PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS — PUC MINAS

DISCIPLINA: TESTE DE SOFTWARE

Refatoração de Testes e Detecção de Test Smells

Diagnóstico estrutural com ESLint, análise sistemática de anti-padrões de testes e refatoração arquitetural baseada no padrão Arrange-Act-Assert (AAA).

Jest Framework | ESLint + eslint-plugin-jest | Qualidade de Código | Padrão AAA

Aluno:	Luis Henrique
Matrícula:	Matrícula do Aluno
Instituição / Curso:	PUC Minas — Engenharia de Software / Ciência da Computação
Repositório Fork:	https://github.com/BGLuis/test-smelly
Repositório Base:	https://github.com/CleitonSilvaT/test-smelly

BELO HORIZONTE — 2026

1 Análise Detalhada dos Test Smells Identificados

Na avaliação da suíte de testes original presente em `test/userService.smelly.test.js`, constatou-se que todos os testes passavam na execução inicial do Jest. No entanto, o código apresentava severos **Test Smells** que comprometiam a clareza, o determinismo e a eficácia na detecção de bugs reais. A seguir, analisam-se três dos principais smells encontrados:

1.1. Lógica Condicional e Asserção Condicional (Conditional Test Logic / Conditional Expect)

O que é e onde ocorre: Ocorre no teste `deve desativar usuários se eles não forem administradores`. O autor utilizou um laço `for (const user of todosOsUsuarios)` acoplado a um desvio condicional `if (!user.isAdmin) { ... } else { ... }`, invocando `expect` dentro desses blocos ramificados.

Por que é um mau cheiro: Testes unitários devem ser estritamente lineares e determinísticos. Estruturas de controle de fluxo adicionam complexidade ciclomática ao teste, transferindo a ele a responsabilidade de tomar decisões lógicas que deveriam pertencer ao código sob teste.

Risco que representa: Alto risco de *falsos positivos* e *asserções silenciosamente ignoradas*. Se a lista estiver vazia ou mutada, os blocos `expect` podem nunca ser executados, e o Jest reportará sucesso sem validar nada. O linter flagra isso como erro em `jest/no-conditional-expect`.

1.2. Exceção Silenciosa e Falha Oculta (Silent Catch / Missing Assertion on Failure)

O que é e onde ocorre: Ocorre no teste `deve falhar ao criar usuário menor de idade`. O teste envolve a chamada em um bloco `try { userService.createUser(...) } catch (e) { expect(e.message)... }`, sem garantia de que a exceção foi lançada.

Por que é um mau cheiro: O bloco `catch` só executa se o erro ocorrer. Se a validação for removida ou falhar silenciosamente, o fluxo ignora o `catch` e o teste é concluído em verde sem acusar a quebra da regra de negócio.

Risco que representa: Falso positivo crônico, mascarando regressões críticas. Viola o princípio de falhar ruidosamente. O ESLint detecta essa estrutura incorreta através da regra `jest/no-conditional-expect`.

1.3. Teste Ansioso e Múltiplas Responsabilidades (Eager Test / Assertion Roulette)

O que é e onde ocorre: No teste `deve criar e buscar um usuário corretamente`, dois casos de uso distintos são testados no mesmo bloco (criação e busca), com 4 asserções acumuladas e dependência do fixture global `dadosUsuarioPadrao` (Convidado Misterioso).

Por que é um mau cheiro: Viola a Responsabilidade Única (SRP). Um teste deve validar apenas um comportamento por cenário.

Risco que representa: Roleta de asserções (*Assertion Roulette*). Em caso de falha, é difícil diagnosticar de imediato se o defeito está na persistência da criação ou no algoritmo de busca, elevando o custo de manutenção.

Test Smell	Regra ESLint Associada	Severidade	Impacto Principal
Conditional Expect	`jest/no-conditional-expect`	Erro	Asserções ignoradas silenciosamente sob ramificações de fluxo
Silent Catch	`jest/no-conditional-expect`	Erro	Falso positivo caso a exceção esperada não seja lançada
Disabled Test	`jest/no-disabled-tests`	Aviso	Funcionalidades sem validação ativa (TODO órfão na suíte)
Ghost Test	`jest/expect-expect`	Aviso	Testes vazios que executam sem validar nenhum resultado

2 Processo de Refatoração Arquitetural e Aplicação do Padrão AAA

Para solucionar os problemas diagnosticados, elaborou-se o arquivo `__tests__/userService.clean.test.js`. O processo foi conduzido sob o padrão **Arrange, Act, Assert (AAA)**, conferindo a cada teste foco, independência e legibilidade.

2.1. Estudo de Caso: Refatoração do Teste de Desativação (Eliminação de Loops e Condicionais)

× ANTES: USERSERVICE.SMELLY.TEST.JS

```
// Lógica condicional, loop e múltiplos cenários
test('deve desativar usuários se eles...', () => {
  const usuarioComum = userService.createUser(...);
  const usuarioAdmin = userService.createUser(...);
  const todos = [usuarioComum, usuarioAdmin];

  for (const user of todos) {
    const res = userService.deactivateUser(user.id);
    if (!user.isAdmin) {
      // Expect condicional (ESLint Erro!)
      expect(res).toBe(true);
      const u = userService.getUserById(user.id);
      expect(u.status).toBe('inativo');
    } else {
      expect(res).toBe(false);
    }
  }
});
```

✓ DEPOIS: __TESTS__/USERSERVICE.CLEAN.TEST.JS

```
// 1. Caso isolado para Usuário Comum
test('deve desativar com sucesso usuário comum', () => {
  // Arrange
  const user = userService.createUser('Comum', 'c@t.com',
30, false);
  // Act
  const ok = userService.deactivateUser(user.id);
  // Assert
  expect(ok).toBe(true);
  const atual = userService.getUserById(user.id);
  expect(atual.status).toBe('inativo');
});

// 2. Caso isolado para Usuário Administrador
test('não deve desativar usuário administrador', () => {
  // Arrange
  const admin = userService.createUser('Admin',
'a@t.com', 40, true);
  // Act
  const ok = userService.deactivateUser(admin.id);
  // Assert
  expect(ok).toBe(false);
  const atual = userService.getUserById(admin.id);
  expect(atual.status).toBe('ativo');
});
```

2.2. Decisões Técnicas de Refatoração e Justificativas

- **Decomposição em Testes Atômicos:** A separação dos fluxos de usuário comum e administrador permitiu isolar completamente as causas de falha. A suíte passa a documentar explicitamente cada regra de negócio como especificação executável.
- **Eliminação Completa de Estruturas Condicionais:** A erradicação do laço `for` e dos blocos `if/else` garantiu fluxo 100% linear e determinístico, extinguindo todas as 3 ocorrências de `jest/no-conditional-expect` apontadas pelo linter naquele método.
- **Tratamento Idiomático de Exceções com `toThrow`:** Substituiu-se o bloco `try/catch` por `expect(() => userService.createUser(...)).toThrow('O usuário deve ser maior de idade.')`. Se o método não disparar o erro esperado, o Jest acusa falha imediatamente.
- **Resiliência a Mudanças Estéticas (Anti-Fragile):** No relatório de usuários, as asserções passaram a validar a presença semântica dos dados essenciais (IDs, nomes e status) e do cabeçalho funcional, deixando de quebrar por variações de espaçamento ou ordenação.
- **Reativação e Implementação de Testes Pendentes:** O teste marcado como `test.skip` foi reativado e implementado com asserções reais, validando o comportamento de relatório com base de dados vazia.

3 Relatório da Ferramenta Estática e Conclusão

3.1. Execução Inicial do ESLint no Código Original (Detecção Automatizada)

Após a configuração das regras no arquivo `.eslintrc.json` com o plugin `eslint-plugin-jest`, a execução do comando `npx eslint .` identificou **6 problemas (4 erros e 2 avisos)** na suíte `test/userService.smelly.test.js`:

```
/home/luis/Documents/puc/testes/test-smell/test/userService.smelly.test.js
  44:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  46:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  49:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  73:7  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  77:3  warning  Tests should not be skipped            jest/no-disabled-tests
  77:3  warning  Test has no assertions                 jest/expect-expect

✖ 6 problems (4 errors, 2 warnings)
```

Como a ferramenta automatizou a detecção: O linter constrói a Árvore Sintática Abstrata (AST) do arquivo de teste. Ao inspecionar os nós onde a função `expect()` é invocada, o plugin verifica se o nó ancestral direto pertence a uma estrutura de controle (como `IfStatement`, `ForStatement` ou `CatchClause`). Caso positivo, sinaliza o smell instantaneamente com a linha e coluna exatas, sem executar o código.

3.2. Validação da Suíte Refatorada

Na validação final, a suíte limpa obteve conformidade plena em ambas as ferramentas:

> **✓ Conformidade Total e Zero Violações no Linter**
>
> Execução de `npx eslint __tests__/userService.clean.test.js`: **0 erros, 0 avisos (código 100% limpo)**.
> Execução de `npm test`: **2 test suites executadas com 100% de aprovação (14 testes passando)**.

3.3. Conclusão e Reflexão

A realização desta atividade evidencia que uma suíte de testes que atinge 100% de passagem ("testes verdes") não é garantia suficiente de qualidade de software. Testes que contêm *Test Smells* — como asserções condicionais, captura silenciosa de exceções e acoplamento frágil — introduzem fragilidade arquitetural e falsa confiança na equipe de engenharia.

A adoção de ferramentas de análise estática como o ESLint integrada a plugins de teste (`eslint-plugin-jest`) atua como um *guard-rail* fundamental no pipeline de Integração Contínua (CI). Essa abordagem automatizada impede que más práticas de teste sejam integradas ao repositório principal, assegurando que os testes permaneçam limpos, modulares, fáceis de manter e verdadeiramente eficazes na detecção precoce de regressões e defeitos de software.